LAB ASSIGNMENT Nº3

NB-IOT-BASED ULTRASONIC DETECTOR

(MKR NB 1500, MEO NB-IOT, FIREBASE, ANDROID STUDIO, MQTT)

1 INTRODUCTION

This assignment will introduce the students to the Narrowband IoT (NB-IoT) technology, which is a 4G/5G extension to support applications requiring very low data rates and low power consumption. Like Lab Assignment Nº 2, it builds upon the Lab Assignment Nº 1 in terms of cloud storage and user interface, requiring no further changes to the Firebase project and Android app. Besides, NB-IoT supports IP stack protocols, namely UDP and TCP, which allows the devices to communicate directly with any machine in the Internet.

The instructions in this assignment are abridged so that, at each step, the students will have to investigate how to implement the requested functionality. At the end of this document, a list of useful references is provided.

2 SETTING-UP, DEPLOYING AND TESTING THE NB-IOT DEVICE AND APPLICATION

The device used in this assignment will perform the same tasks as the one in Lab Assignment Nº 1, but with a different MCU and communication technology. In fact, the sensor and actuator will be the same. The MCU will now consist of an MKR NB 1500, which supports Arduino connectivity using the NB-IoT and LTE-M technologies. In this assignment, the device was previously configured to use NB-IoT. The pinout of MKR NB 1500 is depicted in Figure 1.

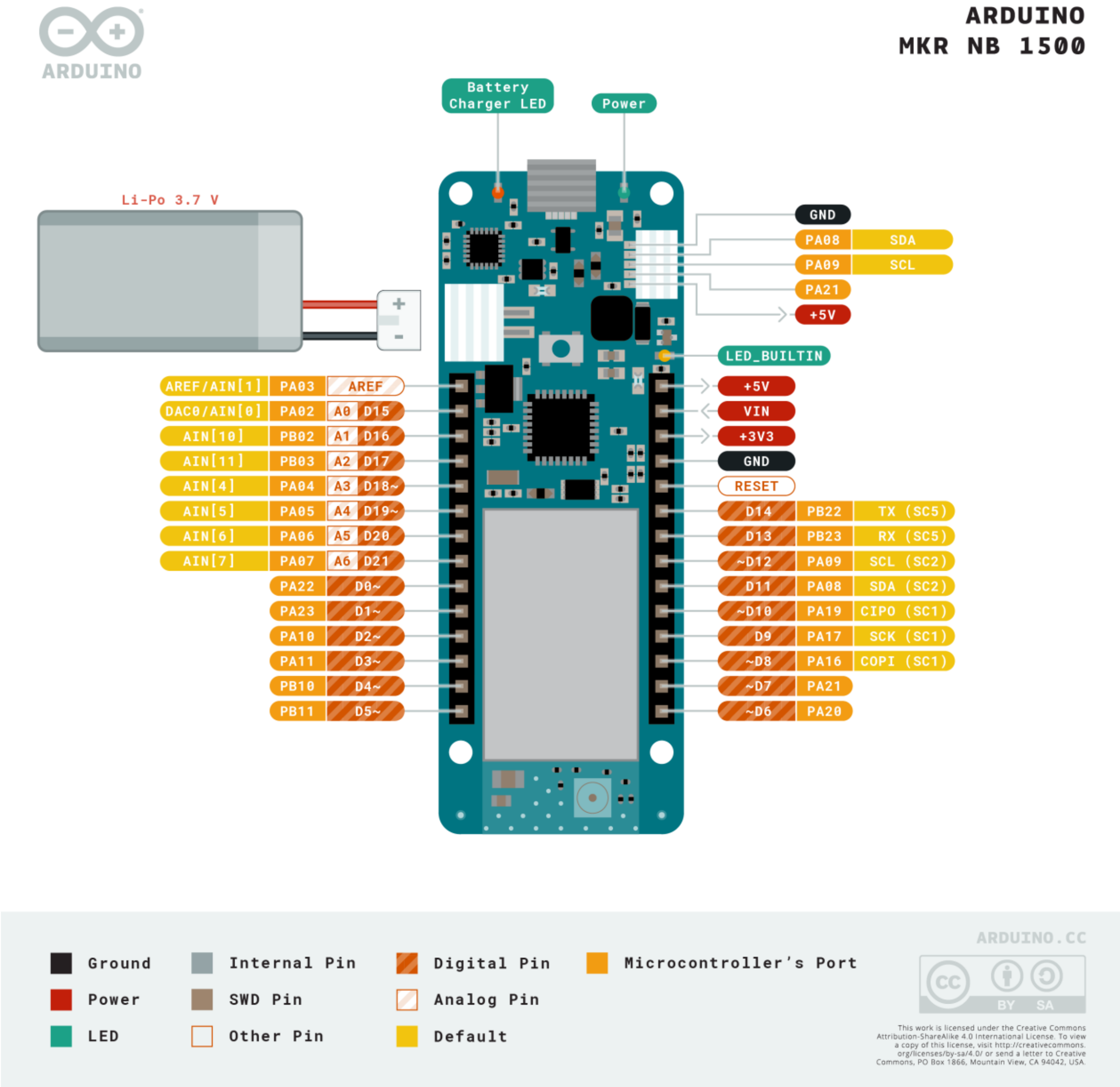

Figure 1: Pinout of MKR NB 1500.

The MKR NB 1500 operates at 3.3 V. The board's main processor is a low power Arm® Cortex®-M0 32-bit SAMD21, like in the other boards within the Arduino MKR family. It includes a SIM socket. A SIM card is required to connect to the mobile network. The NB-IoT connectivity is performed with a module from u-blox, the SARA-R410M-02B, a low power chipset operating in the different bands of the IoT LTE cellular range. It comes with a micro U.FL connector for external antenna coupling. It can optionally be coupled to a 3.7 V battery for standalone operation.

This board presents Arduino type connectors, making it easier to interface with sensors and actuators in the same way as would be done for other boards of the family. This will be used to connect to the HC-SR04 ultrasonic sensor and the piezo buzzer (actuator).

1) Mount the circuit as depicted in Figure 2.

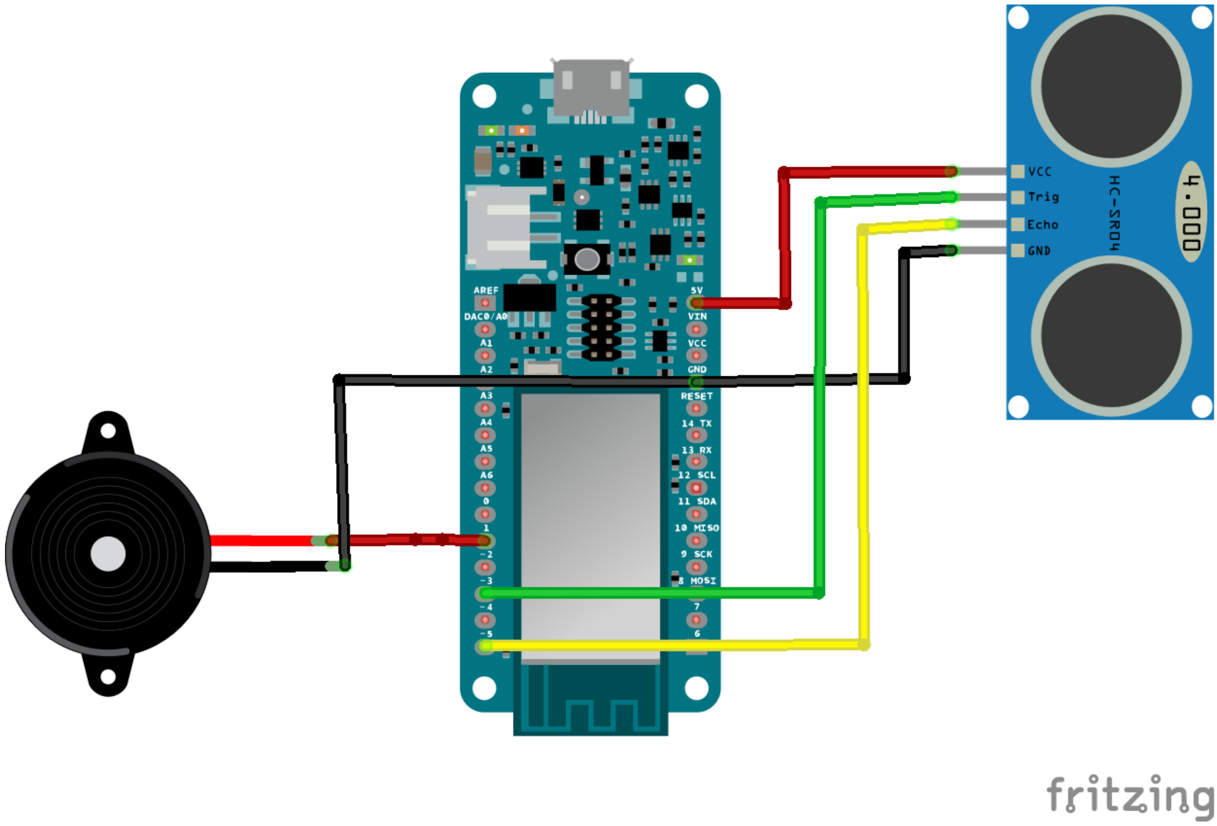

Figure 2: Ultrasonic detection and alarm circuit using Arduino MKR NB 1500.

2) In the Boards Manager, install board Arduino SAMD Boards (32-bit ARM Cortex-M0+).
3) In Manage Libraries, install the MKRNB and PubSubClient libraries.
4) The *mkr_nb_1500_mqtt.ino* code implements functionality that is like Assignment nº 2, but at this time the MKR NB 1500 will directly send/receive data to/from an MQTT server (see below) through the network of the 4G mobile operator.
5) Edit the *mkr_nb_1500_mqtt.ino* code and modify all instances of the string "/RMIC_G00/" so that "00" is replaced by the number of your group.
6) Like in Assignment nº 2, in order to be able to communicate with Google Firebase, a Python script must perform the bridging.

3 INTEGRATING WITH FIREBASE THROUGH MQTT

The 4G mobile network will only provide support for TCP/IP or UDP/IP connectivity between user application modules, it will not provide persistent storage for the data. Consequently, an approach similar to the one followed in Assignment 2 will be followed: the uplink data will be sent to Google Firebase for permanent storage. In this project, the NB-IoT node will send the data to the *test.mosquitto.org* MQTT server, which is available for free. A Python script running anywhere in the Internet (e.g., student's laptop) will also connect to the MQTT server as a client, playing the role of publish/subscribe gateway between MQTT and Google Firebase. In order to achieve this functionality, take the following steps:

1) Use the same virtual environment as for Assignment 2 in order to run the **firebase_mqtt_mosquitto.py** script.

2) Edit the **firebase_mqtt_mosquitto.py** script to change the Google Firebase configuration and MQTT topic:
    a. Change the *firebase_config* structure according to your Google Firebase account data
        i. The API Key can be found in the General separator of Project Settings ⚙;
        ii. The Authorized Domain can be found under the Authentication options, in the Settings separator;
        iii. The Database URL can be found in the Realtime Database options;
        iv. The Storage Bucket URL is not required and can be left as an empty string.
    b. Modify all instances of the string "/RMIC_G00/" so that "00" is replaced by the number of your group.
3) You can now run **firebase_mqtt_mosquitto.py** in any computer connected to the Internet.

4 REFERENCES

[1] The Mosquitto Test Server, https://test.mosquitto.org/ .
[2] The Python Tutorial, 12. Virtual Environments and Packages, https://docs.python.org/3/tutorial/venv.html .